

Trabajo Práctico N.º 2

Representación vectorial de texto: embeddings y búsqueda semántica

Unidad 2 — Representación Vectorial de Texto Modalidad: mismos grupos que tp1, mismo repo **Fecha de entrega:**

1. Contexto

En el TP1 construyeron un corpus propio scrapeando un catálogo de libros, y lo representaron con **TF-IDF**: un vector del tamaño del vocabulario, casi todo ceros, donde cada dimensión corresponde a una palabra. Es el último de los cuatro métodos de codificación de la Unidad 2 —one-hot, Count Vectorizer, TF-IDF y Hash Vectorizer— y comparte con los otros tres la limitación con la que cierra esa sección: **ninguno captura la semántica**.

Esa representación tiene un techo que conviene ver con un ejemplo:

Para TF-IDF, *"una novela sobre piratas"* y *"un relato de bucaneros"* tienen similitud **exactamente cero**. No comparten ningún token.

Un buscador construido así falla ante cualquier usuario que no adivine las palabras exactas del texto. Este TP trabaja sobre esa limitación: representar el **significado** en un espacio denso, donde la cercanía deja de depender de que las palabras coincidan.

Y, sobre todo, trabaja sobre la pregunta que sigue: **¿cómo saben que la solución nueva es mejor que la vieja?**

2. Objetivos de aprendizaje

Al terminar el TP deberían poder:

1. Entrenar embeddings de palabra sobre un corpus propio y explicar qué optimiza el modelo.
2. Usar un modelo de oración pre-entrenado y justificar cuándo conviene sobre uno propio.
3. Construir vectores de documento y reconocer las pérdidas de información que eso implica.
4. Persistir vectores en una base relacional e indexarlos para búsqueda por similitud.

5. **Diseñar una evaluación con línea de base** y discutir honestamente qué mide y qué no.

El punto 5 es el que más pesa. Todo lo demás es infraestructura para poder llegar ahí.

3. Punto de partida

Necesitan tener funcionando:

- El corpus del TP1 cargado en la tabla **books** de su csv (entre 100-200 libros).
 - La extensión **pgvector** habilitada y verificada.(por el momento solo utilizan el csv y el DataFrame en colab)
-

4. Consigna — parte obligatoria

Primer análisis del corpus

Ya tenemos texto. Primeras decisiones de preprocesamiento — cada una es una ****pérdida de información deliberada****:

- ¿Minúsculas? Perdemos entidades nombradas (`Madrid` vs `madrid`).
- ¿Sacar acentos? Perdemos `sí`/`sí`, `el`/`él`.
- ¿Puntuación? Perdemos límites de oración.
- ¿Stopwords? Perdemos negaciones y relaciones sintácticas.

No hay respuesta universal: depende de la tarea de aguas abajo.

TF-IDF: que distingue a cada sinopsis del resto del corpus?

1. Con 12 documentos, TF-IDF es casi ruido. ¿Cuántas sinopsis harían falta para que los términos característicos sean estables? ¿Cómo lo medirían?
2. Las sinopsis son texto ****promocional****: escrito para vender, no para describir.

```
¿Qué sesgo introduce eso si entrenamos un clasificador de género literario?
3. Un libro tiene varios géneros (`Aventuras`, `Histórico`, `Intriga`...). Eso hace de la clasificación un problema **multi-etiqueta**. ¿Cómo cambia la evaluación respecto de multi-clase?
4. ¿Qué pasa con los libros en gallego o catalán que aparecen en el catálogo?
¿Detección de idioma antes de tokenizar?
```

A. Recuperar y preparar el corpus

Traigan los documentos desde Postgres (por ahora desde el CSV) y preparen **dos versiones** del texto: una tokenizada y limpia, y otra cruda.

Atención. La limpieza que usaron para TF-IDF —minúsculas, sin stopwords, sin puntuación— es correcta para bag-of-words y **destructiva** para un modelo de oración, que fue entrenado sobre texto natural y usa el orden y las palabras funcionales. El preprocesamiento pertenece al modelo, no al corpus. Documenten esta decisión.

B. Word2Vec / FastText: propio contra pre-entrenado

En la Unidad 2 **cargaron** el modelo pre-entrenado del Spanish Billion Word Corpus (**SBW-vectors-300-min5**). Acá van a hacer las dos cosas y compararlas. Reporten:

- Los parámetros elegidos para el modelo propio (**vector_size, window, min_count, sg**) **con su justificación**.
- Vecinos más cercanos de al menos 4 palabras del dominio, **en los dos modelos, lado a lado**.
- Cómo construyeron el vector de documento a partir de los vectores de palabra.

Sobre ese último punto: la Unidad 2 ya mostró, con spaCy, que promediar vectores de palabra hace que *"this is cool"* e *"is this cool"* tengan similitud 1.0. El promedio descarta el orden. Digan qué más se pierde, y por qué lo usan igual.

C. Modelo de oración: SBERT

Generen embeddings con un modelo de oración multilingüe. Arranquen con `distiluse-base-multilingual-cased-v1`, el de la Unidad 2; si prueban otro de los que aparecen en la sección de MTEB del apunte, justifiquen por qué.

Reporten la dimensión, el límite de tokens del modelo y **cuántos de sus documentos se truncan** por superarlo. Ese último dato el modelo no lo avisa: hay que ir a buscarlo.

D. Comparación y visualización

Con las mismas consultas, comparen los espacios generados. Incluyan como mínimo:

- Un ranking lado a lado para al menos 3 consultas en lenguaje natural.
- La **distribución de similitudes entre pares aleatorios** de cada modelo (media, desvío, rango). Un espacio donde todo se parece a todo no discrimina, aunque el ranking devuelva resultados.
- Una **proyección 2D** (PCA o t-SNE) de los documentos, coloreada por género. La Unidad 2 proyectó ocho palabras sueltas; acá son cientos de documentos y el color es una etiqueta que el modelo nunca vio.

Si usan t-SNE, digan qué `perplexity` eligieron y por qué **no** se pueden interpretar las distancias entre clusters. Si usan PCA, informen la varianza explicada por las dos componentes: proyectar 512 dimensiones sobre 2 descarta casi todo, y una nube que parece mezclada puede estar perfectamente separada en el espacio original.

E. Persistencia e indexado

Guarden los vectores de **al menos dos modelos** en Postgres:

- Una tabla por modelo, con la dimensión correspondiente. Expliquen por qué no pueden convivir dos dimensiones distintas en la misma columna indexada.
- Índice **HNSW** con la `opclass` que corresponde al operador que van a usar.
- Manejo explícito de vectores nulos o no finitos **antes** de insertar.

F. Búsqueda semántica

Implementen una función `buscar(consulta, k)` que resuelva la similitud **en SQL**, no en Python, y que permita combinar similitud con un filtro por metadata (por ejemplo, género).

Expliquen qué problema de *recall* aparece al combinar un índice aproximado con un filtro selectivo, y qué harían al respecto.

5. Evaluación de los modelos (núcleo del TP)

Hasta acá tienen un buscador que devuelve resultados plausibles. Eso no alcanza: `argsort` ordena ruido con la misma prolijidad con que ordena señal.

Construyan un conjunto de evaluación propio: mínimo **10 consultas** en lenguaje natural, y para cada una, los libros de *su* corpus que consideran relevantes. Se entrega como `queries.json`. Es la parte más valiosa del TP y la única que no puede automatizarse.

Con ese conjunto, calculen **precision@k** para:

- TF-IDF (línea de base léxica, la del TP1)
- Cada modelo de embeddings
- **Elegir al azar** (piso)

El piso no es un trámite. Si el azar da 0.45 porque un género cubre media colección, entonces 0.55 apenas supera el azar aunque suene bien. **Toda métrica necesita su línea de base.**

Incluyan además al menos una consulta diseñada para que **no comparta ninguna palabra** con los documentos relevantes. Es el caso donde TF-IDF no puede ganar por construcción, y donde la diferencia entre las dos familias de representación se vuelve visible.

Es un resultado perfectamente aceptable —y frecuente— que TF-IDF empate o gane en varias consultas. Si eso pasa, la consigna **no** es cambiar la métrica hasta que dé lo esperado: es explicar por qué pasa.

6. Parte avanzada (elegir una)

Una hecha con profundidad vale más que tres esbozadas.

Clustering. K-Means sobre los embeddings, comparado contra los géneros reales con ARI o V-measure. ¿El espacio reconstruye la taxonomía sin haberla visto? Discutan por qué un problema multi-etiqueta penaliza a un clustering duro.

Recomendación. "Más libros como este", usando un libro como consulta. La decisión interesante es **qué excluir**: un recomendador que ante el tomo 1 devuelve los tomos 2 a 7 es correcto e inútil.

Búsqueda híbrida. Combinar el ranking vectorial con la búsqueda full-text nativa de Postgres (`ts_rank`) mediante *Reciprocal Rank Fusion*. Midan si le gana a cada una por separado.

RAG. Recuperar los k documentos más relevantes y armar el prompt para un modelo generativo. El foco está en el diseño del prompt —prohibir explícitamente inventar, exigir citas— y en detectar alucinaciones, no en la generación en sí.

Chunking. Partir las sinopsis largas y guardar varios vectores por libro. ¿Mejora el recall en los documentos que hoy se truncan?

Doc2Vec. Es el único modelo de la Unidad 2 que entrena vectores *de documento* en vez de promediar palabras. ¿Le gana al promedio sobre el mismo corpus? Ojo con `infer_vector()`, que es estocástico: dos llamadas con el mismo texto no devuelven el mismo vector.

7. Entregables

#	Archivo	Contenido
1	TP2_apellido1_apellido2.ipynb	Notebook ejecutado, con salidas visibles
2	queries.json	Conjunto de evaluación propio (mín. 10 consultas)
3	informe.pdf	Máximo 3 páginas
4	—	Base poblada y accesible en Supabase

Las dos tablas de embeddings tienen que ser de **familias distintas**: un promedio de word vectors y un modelo de oración. Dos variantes de lo mismo no cuentan.

Contenido obligatorio del informe

1. ¿Qué modelo elegirían para producción y por qué? El criterio no puede ser sólo la métrica: consideren costo, privacidad, reproducibilidad y dependencia de terceros.
2. ¿Cuánto mejor es que la línea de base TF-IDF? ¿En qué tipo de consulta gana y en cuál no?
3. ¿Qué mide y qué **no** mide la métrica que usaron?
4. **Un caso concreto donde la búsqueda falló**, con su hipótesis del porqué. Sin este punto el informe está incompleto.

Requisitos técnicos

- ☐ Word2Vec propio comparado contra `SBW-vectors-300-min5`, con vecinos lado a lado
- ☐ Vectores normalizados, con manejo explícito de vectores nulos o `NaN`

- ☐ La *opclass* del índice corresponde al operador de distancia usado
 - ☐ Una proyección 2D del corpus, con su advertencia metodológica
 - ☐ Búsqueda que combine similitud y filtro por metadata
 - ☐ **precision@k** sobre su set manual, con TF-IDF y el piso de azar al lado
 - ☐ Credenciales fuera del notebook
-

8. Criterios de evaluación

Criterio	Peso
Rigor de la evaluación: líneas de base, límites de la métrica, interpretación	30 %
Corrección del pipeline de embeddings y su persistencia	25 %
Uso de pgvector: esquema, índices, consultas	20 %
Parte avanzada	15 %
Claridad del informe	10 %

Sobre la nota. Un notebook que corre perfecto con un informe que dice "SBERT anduvo mejor", sin piso y sin discutir la métrica, saca **menos** que uno con resultados peores pero bien interrogados. Detectar que el propio modelo no está funcionando es una habilidad más difícil —y más útil— que hacerlo andar de casualidad.

Descuentan

- Credenciales de conexión visibles en el notebook o en sus salidas.
 - Métricas sin línea de base.
 - Vectores nulos o **NaN** insertados en la base.
 - Índice construido con una *opclass* que no corresponde al operador usado.
 - Conclusiones que el experimento no sostiene.
-

9. Uso de asistentes de IA

Está permitido y no hace falta esconderlo. Se pide que **declaren** en el informe, en un párrafo, para qué los usaron.

Dos advertencias prácticas. Primero: van a defender el TP oralmente, así que conviene entender lo que entregan. Segundo, más específico de este TP: los asistentes son muy buenos generando pipelines que corren y muy malos advirtiéndole que una métrica no significa lo que parece. La parte que más pesa en la nota es justamente esa.

10. Recomendaciones

- **Empiecen por la infraestructura.** El Anexo A lleva 20 minutos la primera vez. Dejarlo para el final es la forma más común de entregar tarde.
 - **Los proyectos gratuitos de Supabase se pausan a los 7 días sin actividad.** Si no tocan la base durante una semana, la van a encontrar apagada. Se reactiva desde el panel en aproximadamente medio minuto, pero enterarse la noche de la entrega es evitable.
 - **Escriban `queries.json` temprano**, antes de ver los resultados. Definir qué es relevante después de ver qué devolvió el modelo es hacer trampa contra ustedes mismos.
 - **Guarden los embeddings en disco** además de en la base. Recalcularlos cuesta minutos cada vez que se reinicia el entorno de Colab.
 - **No usen el pin de versiones del apunte.** La Unidad 2 fija `numpy==1.23.5` para los ejemplos de gensim; ese numpy no convive con `sentence-transformers`, y este TP usa las dos librerías en el mismo notebook. Va `gensim>=4.3.3` con el numpy que trae Colab.
-

Anexos

- **Anexo A** — Creación de la cuenta y el proyecto en Supabase
- **Notebook guía** — `TP2_embeddings_busqueda_semantica.ipynb`
- **Teoría** — Unidad 2, *Representación Vectorial de Texto*

Qué agrega este TP a la Unidad 2

El apunte cubre los modelos y la similitud de coseno. Lo nuevo acá:

Tema	Dónde
Negative sampling: por qué el softmax sobre todo el vocabulario es inviable	Parte B
Persistencia de vectores en Postgres con <code>pgvector</code>	Parte E

Tema	Dónde
Índices aproximados (HNSW), recall, opclass según el operador	Parte E
Filtrar por metadata y buscar por similitud en la misma consulta	Parte F
Evaluación con línea de base y piso de azar	Parte 5